4장 커밋

깃을 사용하여 개발 중인 소스 코드의 이력을 만들 수 있습니다. 그중 가장 기본인 커밋을 알아보겠습니다. 커밋은 깃에서 '코드의 변화를 기록하는 것'을 의미합니다.

4.1 코드의 변화

4.2 새 파일 생성 및 감지

4.3 깃에 새 파일 등록

4.4 첫 번째 커밋

4.5 커밋 확인

4.6 두 번째 커밋

4.7 메시지가 없는 빈 커밋

4.8 커밋 아이디

4.9 커밋 로그

4.10 diff 명령어

4.11 정리

4.1 코드의 변화

깃은 개발 중인 코드의 이력을 만들 수 있습니다. 깃이 코드 변화를 기록하는 것을 커밋(commit)이라고 합니다. 먼저 커밋의 뜻부터 알아봅시다. 영어로 commit은 여러 의미가 있습니다. 그중 깃의 동작과 가장 유사한 의미는 '~를 적어 두다'입니다. 즉, 커밋은 의미 있는 변경 작업들을 저장소에 기록하는 동작입니다.

개발 과정에서 소스 코드는 수없이 수정됩니다. 일반적으로는 새로운 기능을 추가하는 코드를 삽입합니다. 또 버그를 수정하려고 많은 코드를 이동하거나 대체합니다. 이러한 코드 수정은 개발 목적을 달성하는 작업들입니다.

예를 들어 다음과 같이 텍스트 코드를 수정한다고 합시다.

변경 전		변경 후
안녕하세요. 반갑습니다.	>	안녕하세요. 지니입니다. 이렇게 만나서 반갑습니다.

작업자는 개발 과정에서는 수정 내용을 기억하지만, 변경된 내용이 많거나 시간이 흐르면 이를 모두 기억하기 어렵습니다. 개발하는 도중 실수나 여러 가지 이유로 변경 전 시점으로 되돌아가야 한다면 어떻게 해야 할까요?

4.1.1 파일 관리 방법

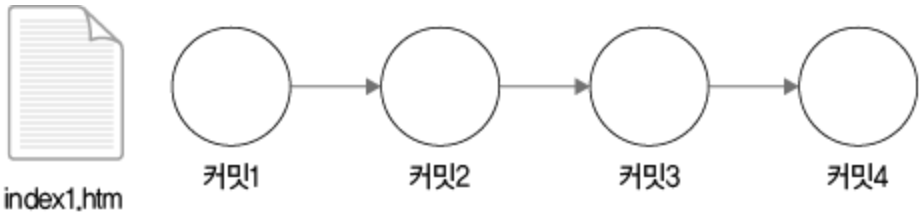
먼저 깃이 없던 시절, 전통적인 파일의 이력 관리 방법을 알아봅시다. 보통 우리는 의미 있는 변경을 할 때 파일을 복사합니다. 그리고 복사한 새 파일에는 추가하거나 변경하고 싶은 내용을 적용합니다. 하지만 이렇게 파일을 복사하는 형태는 파일의 변경 내역을 기록하는 것보다 더 많은 파일을 생성하고 관리해야 하는 부작용이 있습니다. 또 모든 내용이 중복되기 때문에 용량도 많이 차지합니다.

▼ 그림 4-1 파일 복사로 파일 관리



반면 깃의 커밋은 새로 변경된 부분만 추출하여 저장합니다. 그것도 파일 이름을 변경하지 않고도 동일한 파일 이름으로 하나로 관리가 가능합니다. 즉, 시간에 따라 변화되는 내용만 관리하고, 코드가 변화된 시간 순서에 따라서 영구적으로 저장합니다. 이를 커밋이라고 합니다.

▼ 그림 4-2 깃으로 파일 관리



개발자 입장에서는 복잡한 구조의 파일을 관리하지 않아도 되고, 여러 개의 파일보다는 파일 하나로 모든 이력을 처리하기 때문에 유용합니다. 커밋은 부모 커밋(parent commit)을 기반으로 변화된 부분만 새로운 커밋으로 생성합니다. 그리고 커밋은 파일의 시간적 변화도 함께 저장합니다.

4.2 새 파일 생성 및 감지

커밋을 직접 실습해 봅시다. 워킹 디렉터리에 새로운 파일을 하나 생성하여 커밋 과정을 따라가 보겠습니다.

4.2.1 새 파일 생성

실습을 위해 간단한 HTML 파일을 하나 작성합니다. 에디터를 이용하여 코드를 작성하면 됩니다. 필자는 VS Code를 이용하겠습니다.

```
$ mkdir gitstudy04 새 폴더 만들기


$ cd gitstudy04 만든 폴더로 이동


$ git init 저장소를 깃으로 초기화


Initialized empty Git repository in E:/gitstudy04/.git/




infoh@hojin MINGW64 /e/gitstudy04 (main)


$ code index.htm VS Code를 사용하여 파일 작성
```

index.htm은 다음과 같이 작성합니다.

index.htm

```
<!DOCTYPE html>

<html>

<head>

    <meta charset="utf-8" />

    <meta name="viewport" content="width=device-width, initial-scale=1">

    <title>Page Title</title>

</head>

<body>




</body>

</html>
```

4.2.2 깃에서 새 파일 생성 확인

워킹 디렉터리에 새 파일이 생성되었습니다. 워킹 디렉터리에 새 파일이 추가되면 깃은 변화된 상태를 자동으로 감지합니다. 이때 깃 상태를 확인할 수 있는 명령어가 status입니다.

status 명령어를 입력하면 메시지가 출력되는 것을 볼 수 있습니다.

```
infoh@hojin MINGW64 /e/gitstudy04 (main)


$ git status 상태 확인


On branch main


No commits yet




Untracked files:


  (use "git add <file>..." to include in what will be committed)

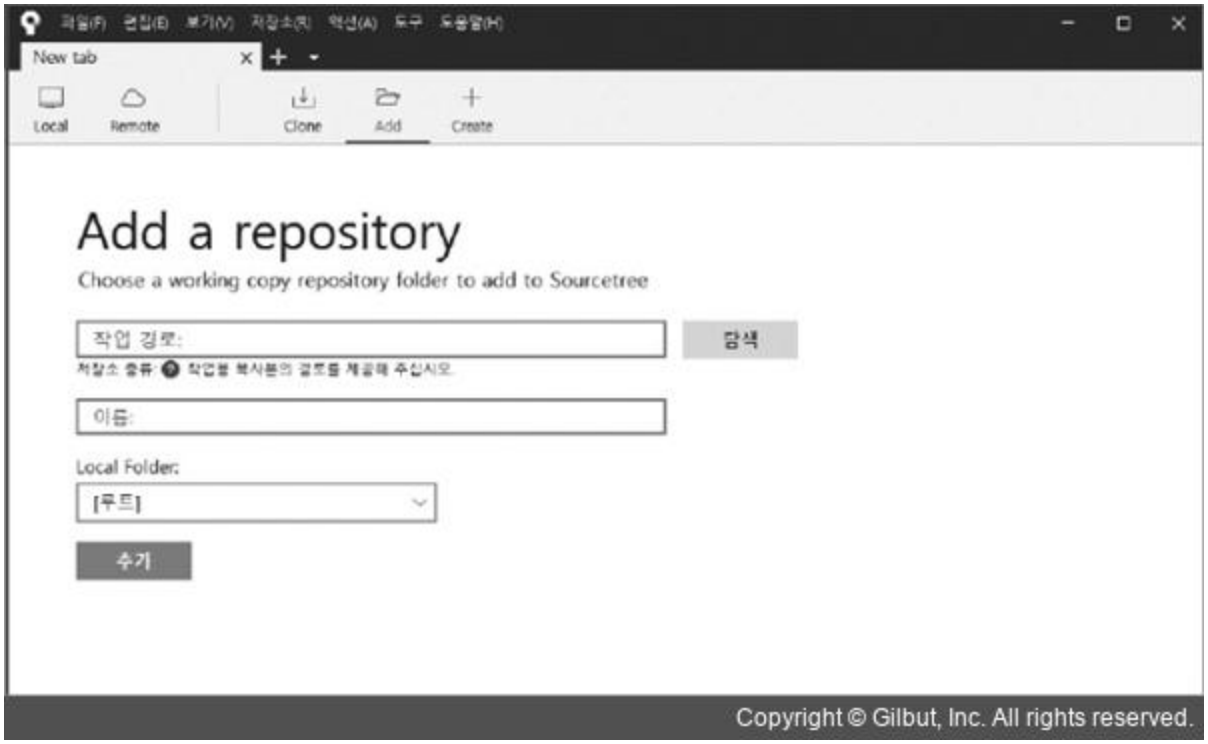

 index.htm 새로운 파일이 등록된 것을 확인


nothing added to commit but untracked files present (use "git add" to track)
```

4.2.3 소스트리에서 새 파일 감지

소스트리를 사용하여 깃의 status 명령어와 동일한 상태를 확인할 수 있습니다. 소스트리를 실행합니다. 아직 gitstudy04 폴더와 소스트리를 연동하지 않았습니다. 새 탭에서 Add 버튼을 클릭합니다. 탐색을 눌러 깃 배시에서 만든 gitstudy04 폴더를 찾아 선택한 후 추가를 누릅니다.

▼ 그림 4-4 소스트리에 저장소 추가

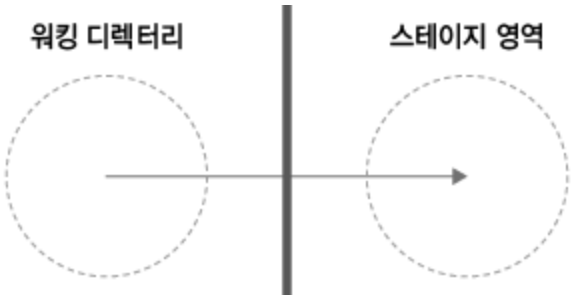


4.3 깃에 새 파일 등록

깃의 워킹 디렉터리에 새 파일이 생성되었습니다. 워킹 디렉터리에 있는 파일은 깃이 자동으로 추적 관리하지 않습니다. 커밋을 하려면 파일의 상태가 추적 가능해야 합니다.

워킹 디렉터리에 새로 추가된 untracked 상태의 파일을 추적 가능 상태로 변경하는 것을 등록이라고 합니다. 또 파일을 등록하면 워킹 디렉터리의 파일이 스테이지 영역에 추가됩니다. 스테이지 영역의 관리 목록에 추가된 파일만 깃에서 이력을 추적할 수 있습니다.

▼ 그림 4-6 새 파일 등록



워킹 디렉터리는 작업을 위한 일종의 샌드박스(서로 분리되어 있는 영역)와 같습니다.

4.3.1 스테이지에 등록

깃에서 등록이란 정확히 무엇을 의미할까요? 등록이란 워킹 디렉터리에 있는 파일을 스테이지(stage) 영역으로 복사하는 것을 의미합니다. 여기서 '복사'는 실제 파일을 복사하는 것을 의미하지는 않습니다. 깃 내부에서 논리적인 기록을 변경하는 과정일 뿐입니다. 복사라고 표현한 것은 이해하기 쉽게 풀어 쓴 것입니다.

워킹 디렉터리에 추가된 모든 파일을 커밋할 때는 반드시 이 과정을 거쳐야 합니다. 그래야 깃에서 버전 이력을 관리할 수 있습니다. 스테이지에 등록되지 않은 unstage 상태의 파일들은 커밋할 수 없습니다. 깃은 커밋하기 전에 파일들이 stage 상태인지 unstage 상태인지를 판단합니다. 스테이지 영역으로 등록된 파일들은 tracked 상태로 자동 변경됩니다.

4.3.2 파일의 추적 상태 확인

워킹 디렉터리에 있는 새로운 파일이 스테이지 영역에 등록되었습니다. 콘솔창에서 status 명령어를 사용하여 등록 상태를 다시 한 번 확인해 보겠습니다.

```
infoh@hojin MINGW64 /e/gitstudy04 (main)


$ git status 상태 확인


On branch main


No commits yet


Changes to be committed:


  (use "git rm --cached <file>..." to unstage)


        new file:   index.htm 스테이지에 등록, 새 파일 상태
```

이전 상태와 달리 new file 메시지가 출력됩니다. 이는 스테이지 영역에 파일을 정상적으로 등록했다는 의미입니다.

앞으로 좀 더 학습해야 하지만, 스테이지 영역에 등록되었다고 커밋이 된 것은 아닙니다. 아직 커밋 명령어를 학습하지 않았습니다. 등록은 커밋을 하기 전 선행 작업일 뿐입니다.

4.3.3 파일 등록 취소

이번에는 tracked 상태의 파일을 untracked 상태로 변경해 보겠습니다. 스테이지에 등록하는 것과 반대 과정입니다. 등록 취소는 워킹 디렉터리와 스테이지 영역을 서로 왔다 갔다 할 수 있는 방법입니다.

unstage 상태로 변경하려면 삭제(rm)나 리셋(reset) 명령어를 사용합니다.

▼ 그림 4-10 스테이지 영역의 파일 등록 취소



rm 명령어로 삭제해 보겠습니다. 스테이지 영역에서만 등록된 파일을 삭제하려고 --cached 옵션을 함께 사용합니다.

```
infoh@hojin MINGW64 /e/gitstudy04 (main)


$ git rm --cached index.htm 스테이지 삭제


rm 'index.htm'
```

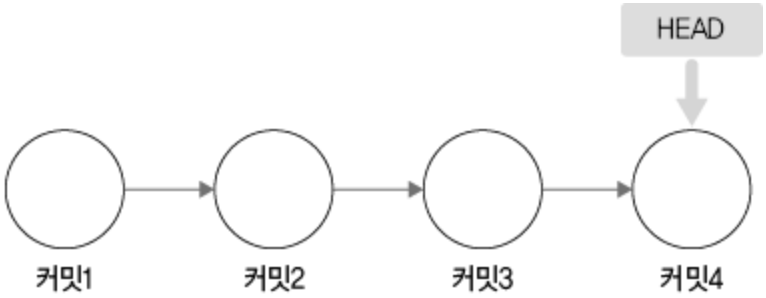
4.4 첫 번째 커밋

워킹 디렉터리에 생성된 파일을 스테이지 영역으로 등록하는 과정을 알아보았습니다. 지금까지 한 작업은 커밋의 준비 작업들입니다. 이번에는 스테이지 영역에 등록된 파일들을 커밋해서 기록하는 과정을 알아보겠습니다.

4.4.1 HEAD

커밋을 학습하기 전에 HEAD 개념을 하나 더 알아봅시다. 깃에는 HEAD라는 포인터 개념이 있습니다. HEAD는 커밋을 가리키는 묵시적 참조 포인터입니다.

▼ 그림 4-11 HEAD



HEAD는 최종적인 커밋 작업의 위치를 가리킵니다. 앞에서 새로운 커밋은 이전 부모 커밋을 기반으로 새로운 커밋을 만든다고 했습니다. HEAD는 바로 부모 커밋을 가리킵니다. 단 깃을 설치하고 처음 커밋할 때는 HEAD의 포인터가 없습니다. 최소한 한 번 이상 커밋을 해야만 HEAD가 존재합니다.

HEAD는 커밋될 때마다 한 단계씩 이동합니다. 그리고 마지막 커밋 위치를 가리킵니다. HEAD는 커밋이 변화한 최종 시점을 의미합니다.

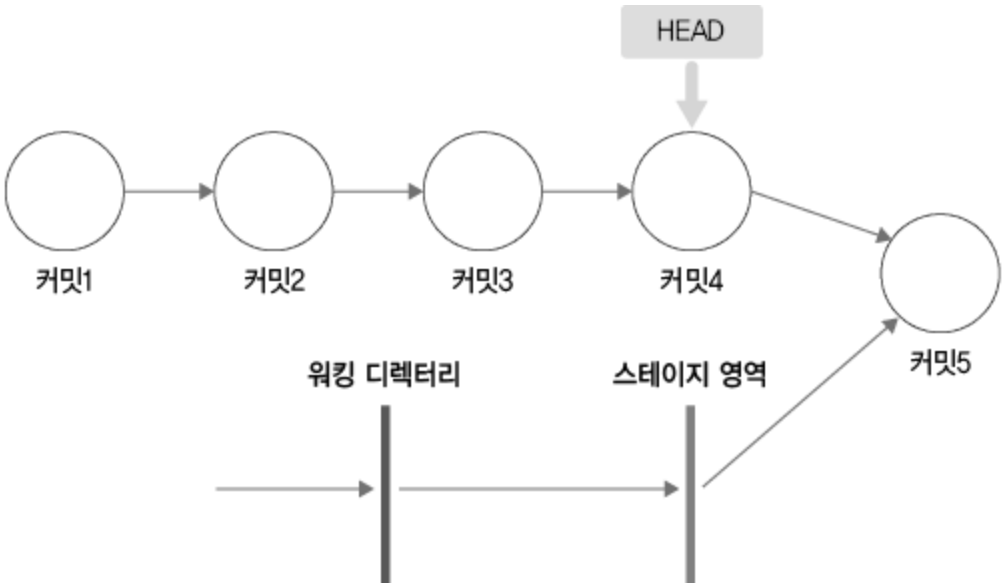
4.4.2 스냅샷

커밋은 파일 변화를 깃 저장소에 영구적으로 기록합니다. 이러한 커밋은 이전에 파일을 복사하여 관리하던 방식과는 큰 차이가 있습니다.

깃이 다른 버전 관리 도구와 다른 점은 스냅샷(snapshot) 방식을 이용한다는 것입니다. 파일을 복사하는 방식으로 수정본을 관리하면 같은 내용을 반복해서 저장하기에 많은 용량을 차지합니다. 또 수정된 부분들을 일일이 찾아야 하기 때문에 검색할 때도 매우 불편합니다.

깃은 이러한 시스템적인 단점을 해결하려고 변경된 파일 전체를 저장하지 않고, 파일에서 변경된 부분을 찾아 수정된 내용만 저장합니다. 마치 변화된 부분만 찾아 사진을 찍는 것과 같다고 하여 스냅샷 방식이라고 합니다.

▼ 그림 4-12 파일에서 변경된 부분을 찾아 사진을 찍듯이 저장



깃의 스냅샷은 HEAD가 가리키는 커밋을 기반으로 사진을 찍습니다. 그리고 이를 스테이지 영역과 비교하여 새로운 커밋으로 기록합니다. 이처럼 깃은 스냅샷 방식을 이용하여 빠르게 버전의 차이점을 처리하고, 용량을 적게 사용합니다.

4.4.3 파일 상태와 커밋

커밋은 변화된 내용을 영구적으로 깃 저장소에 기록합니다. 새롭게 생성된 파일을 커밋하려면 반드시 tracked 상태로 변경해 주어야 합니다. tracked 상태로 파일이 변경됨과 동시에 스테이지 영역에 등록합니다.

tracked 상태인 파일을 수정하면 다시 modified 상태로 변경됩니다. modified는 untracked 상태입니다. untracked 상태의 파일은 반드시 등록 명령으로 다시 스테이지 상태로 재등록해야 합니다. 재등록하면 다시 tracked 상태로 변경됩니다.

커밋하기 전에는 status 명령어로 항상 상태를 확인하는 습관이 필요합니다. 워킹 디렉터리가 깨끗하게 정리되어 있지 않으면 커밋 명령어를 수행할 수 없습니다.

커밋을 하려면 스테이지 영역에 새로운 변경 내용이 있어야 합니다. 수정된 내용이 스테이지 영역으로 등록되지 않으면 커밋을 할 수 없습니다. 커밋은 수정된 내용을 한 번만 등록합니다. 스테이지 영역의 파일이 변경되지 않았다면 커밋을 두 번 실행할 수 없습니다. 깃은 스테이지 영역의 변경된 내용을 기준으로 스냅샷을 만들어 커밋하기 때문입니다.

4.5 커밋 확인

터미널과 소스트리에서 커밋하는 방법을 알아보았습니다. 성공적으로 커밋을 했다면 이를 확인해 보아야 합니다.

4.5.1 스테이지 초기화

먼저 터미널에서 status 명령어를 실행해서 상태를 확인합니다.2

```
infoh@hojin MINGW64 /e/gitstudy04 (main)

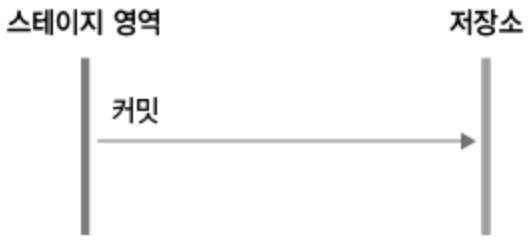

$ git status 상태 확인


On branch main


nothing to commit, working tree clean 워킹 트리 정리됨
```

이전과 달리 working tree clean 메시지를 볼 수 있습니다. 커밋을 하면 스테이지 영역은 초기화됩니다. 더 이상 추가된 새로운 파일과 수정된 파일이 없다는 의미입니다.

▼ 그림 4-16 스테이지 초기화



항상 커밋 전후에 status 명령어로 상태를 확인하는 것이 좋습니다.

4.5.2 로그 기록 확인

커밋한 후 커밋 기록은 어떻게 확인할까요? 깃은 커밋 목록을 확인할 수 있는 log 명령어를 별도로 제공합니다.

$ git log

log 명령어는 시간 순으로 커밋 기록을 출력하는데, 최신 커밋 기록부터 내림차순으로 나열합니다.

```
infoh@hojin MINGW64 /e/gitstudy04 (main)


$ git log


commit e2bce41380691b0a34aeab7db889a6c30fed8287 (HEAD → main)


Author: hojin <infohojin@gmail.com>


Date:   Sat Jan 5 18:24:50 2019 +0900
```

인덱스 페이지 레이아웃

커밋한 후에는 습관적으로 한 번씩 log 명령어를 실행하여 기록을 확인하는 것이 좋습니다. 또 log 명령어는 다양한 커밋 기록을 확인할 수 있도록 여러 옵션을 제공합니다. -help 옵션으로 확인할 수 있습니다.

Note

커밋 시간을 맹목적으로 믿을 수는 없습니다. 깃은 분산형 저장소로 각자의 PC에 설정한 시간 정보를 바탕으로 커밋 기록을 작성합니다. 작업 중인 컴퓨터가 다른 지역의 시간이나 잘못된 시간으로 설정되어 있을 수도 있습니다.

4.5.3 소스트리에서 로그 기록 확인

터미널로 로그 기록을 확인하는 것은 가독성이 좋지 않습니다. 커밋 횟수가 많을수록 보기도 어렵습니다. 이때는 소스트리를 이용하면 좀 더 직관적으로 커밋 기록을 확인할 수 있습니다. 소스트리 같은 GUI 도구를 사용하도록 추천하는 이유이기도 합니다. 소스트리에서 왼쪽의 브랜치 탭을 선택합니다.

▼ 그림 4-17 소스트리에서의 브랜치



앞에서 status 명령어를 실행했을 때 “On branch main” 메시지를 본 적이 있을 것입니다. 또 log 명령어를 실행한 결과에서 (HEAD -> main) 부분도 보았습니다.

브랜치는 나중에 다시 설명하겠지만, 깃을 처음 생성하면 자동으로 main 브랜치 1개를 생성합니다. 또 커밋은 main 브랜치 안에 기록됩니다. 소스트리의 main 브랜치를 선택하면 다음 화면을 볼 수 있습니다. 오른쪽 화면에 커밋의 로그 기록이 순차적으로 나열되는 것을 확인하세요.

▼ 그림 4-18 소스트리에서 브랜치 확인



이렇게 직관적인 뷰 때문에 터미널(깃 배시)과 소스트리를 같이 이용하길 권장합니다.

소스트리 목록은 크게 ‘그래프, 커밋 메시지(설명), 작성자, 날짜, 커밋 아이디’ 다섯 가지로 구분합니다. 이와 관련된 내용은 실습하면서 계속 설명해 나갈 것입니다.

4.6 두 번째 커밋

앞에서 처음으로 새 파일을 생성하고 커밋 작업을 수행했습니다. 앞에서 만들었던 예제(index.htm) 파일은 단순한 HTML 골격이라 화면에 아무 내용도 출력하지 않습니다. 이번에는 내용을 수정하고 수정된 내역을 두 번 커밋하는 실습을 하겠습니다.

4.6.1 파일 수정

index.htm 파일의 <body></body> 태그 안에 <h1> 태그를 추가하여 간단한 인사말을 넣습니다.

```
infoh@hojin MINGW64 /e/gitstudy04 (main)


$ code index.htm  VS Code 실행
```

index.htm

```
<!DOCTYPE html>

<html>

<head>

    <meta charset="utf-8" />

    <meta name="viewport" content="width=device-width, initial-scale=1">

    <title>Page Title</title>

</head>

<body>

    <h1>hello GIT world!</h1>

</body>

</html>
```

간단히 <h1>hello GIT world!</h1> 한 줄만 추가했습니다. 그 외 내용은 기존과 동일합니다. 수정했다면 저장해 주세요.

4.6.2 파일 변경 사항 확인

변경된 파일을 깃이 어떻게 감지하는지 알아봅시다. 터미널에서 status 명령어를 다시 한 번 실행합니다.

```
infoh@hojin MINGW64 /e/gitstudy04 (main)


$ git status 상태 확인


On branch main


Changes not staged for commit:


  (use "git add <file>..." to update what will be committed)


  (use "git checkout -- <file>..." to discard changes in working directory)


 modified:   index.htm 파일 수정


no changes added to commit (use "git add" and/or "git commit -a")
```

이전과 다른 새로운 메시지를 확인할 수 있습니다. 처음 우리가 파일을 생성했을 때는 new file: index.htm 메시지를 보았습니다. 파일을 수정한 후에는 modified: index.htm 메시지가 출력됩니다. 이처럼 깃은 파일을 새롭게 생성했는지, 수정했는지를 알고 있습니다. 정말 똑똑합니다.

4.6.3 수정된 파일 되돌리기

앞에서 살펴보았듯이, 파일을 수정하면 modified 상태로 변경됩니다. 하지만 수정하는 과정에서 파일을 잘못 수정할 수도 있습니다. 파일을 수정 전 상태로 되돌리려면 어떻게 해야 할까요?

깃을 이용하면 수정한 파일을 커밋 전 마지막 내용으로 쉽게 되돌릴 수 있습니다. 바로 이전 커밋으로 되돌리는 명령어는 다음과 같습니다.

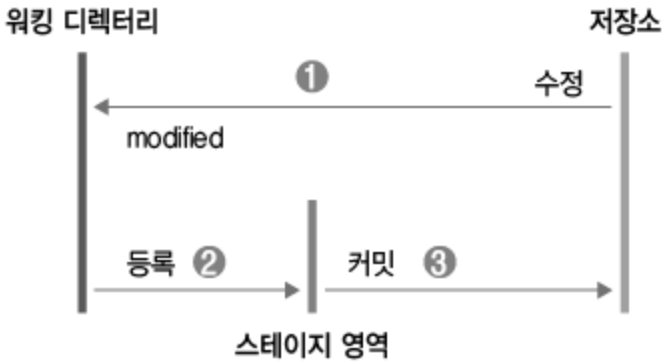
$ git checkout -- 수정파일이름

수정 파일을 되돌리면 이전 커밋 이후에 작업한 수정 내역은 모두 삭제합니다.

4.6.4 스테이지에 등록

변경된 소스 코드를 커밋하는 것은 처음 파일을 생성하고 등록하는 과정과 매우 유사합니다.

▼ 그림 4-20 스테이지에 등록



❶ 기존 파일을 수정하면 해당 파일은 modified 상태로 변경됩니다. 그리고 다시 워킹 디렉터리로 이동합니다. ❷ 파일이 수정되면 반드시 add 명령어로 스테이지 영역에 재등록해야 합니다. 즉, 파일을 수정할 때마다 등록 작업을 반복해야 한다는 것을 잊지 마세요. 또는 소스트리에서 모두 스테이지에 올리기를 사용합니다.

$ git add 수정파일이름

4.6.5 두 번째 커밋

수정된 파일을 커밋할 수 있는 사전 준비 작업을 마쳤습니다. 이번에는 두 번째 커밋을 해 보겠습니다.

그 전에 먼저 커밋 메시지를 좀 더 쉽게 작성하는 방법을 알아봅시다. vi 에디터나 소스트리에서는 커밋 메시지를 여러 줄 작성할 수 있습니다. 이와 달리 커밋과 동시에 간단하게 한 줄짜리 커밋 메시지도 작성할 수 있습니다. 커밋할 때는 -m 옵션을 사용합니다.

$ git commit -m "커밋메시지"

이렇게 커밋 메시지를 작성하면 별도로 vi 에디터를 실행하지 않습니다. -m 옵션 옆에 간단히 입력한 내용을 바로 커밋 메시지로 처리합니다. 다음과 같이 커밋과 동시에 “hello git world 추가”라고 커밋 메시지도 함께 작성해 봅시다.

```
infoh@hojin MINGW64 /e/gitstudy04 (main)


$ git commit -m "hello git world 추가" 커밋 메시지를 같이 입력


[main aa1dd51] hello git world 추가


 1 file changed, 1 insertion(+), 1 deletion(-)
```

실제 작업할 때는 에디터를 여는 대신 간편한 -m 옵션을 많이 사용합니다.

4.6.6 두 번째 커밋 확인

두 번째 커밋 작업을 했습니다. 다시 한 번 로그 기록을 확인해 보겠습니다. 터미널에서 log 명령어를 실행합니다.

```
infoh@hojin MINGW64 /e/gitstudy04 (main)


$ git log 로그 확인


commit aa1dd51a8883b2ea9a54209a00f434a2da01ee85 (HEAD → main)


Author: hojin <infohojin@gmail.com>


Date:   Sat Jan 5 19:31:46 2019 +0900


 hello git world 추가 추가된 커밋 확인




commit e2bce41380691b0a34aeab7db889a6c30fed8287


Date:   Sat Jan 5 18:24:50 2019 +0900


 인덱스 페이지 레이아웃
```

로그 메시지가 좀 더 길어졌습니다. 이전 커밋 로그와 현재 커밋 로그, 이렇게 2개가 출력되었습니다.

4.6.7 깃허브에서 확인

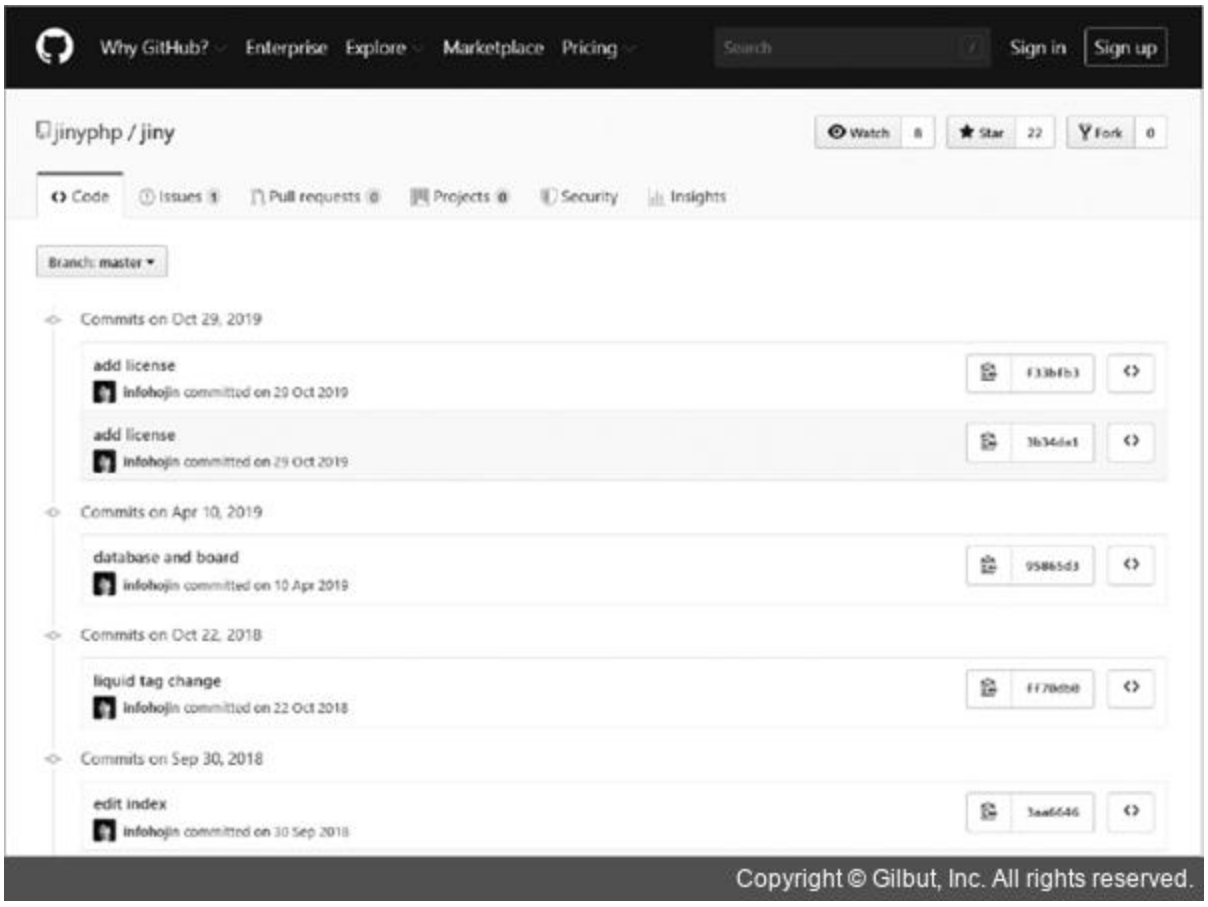
깃은 자신의 컴퓨터뿐만 아니라 원격 저장소를 같이 연동할 수 있습니다. 대표적인 원격 저장소로는 깃허브가 있습니다. 원격 저장소인 깃허브는 커밋된 횟수를 다음과 같이 표시합니다.

▼ 그림 4-22 커밋 횟수 표시



커밋 항목을 클릭하면 다음과 같이 커밋의 상세 목록을 확인할 수 있습니다.

▼ 그림 4-23 깃허브에서 커밋 상세 목록 확인



깃허브와 관련된 내용은 5장에서 자세히 설명하므로, 여기서는 깃허브에서 커밋 횟수와 목록을 확인할 수 있다는 점만 상기하세요.

4.7 메시지가 없는 빈 커밋

커밋을 할 때는 반드시 커밋 메시지를 같이 작성해야 합니다. 하지만 의미가 없는 커밋이라 커밋 메시지를 생략하고 싶을 때도 있습니다. 이러한 특별한 상황에 대비하여 깃은 메시지가 없는 커밋 작성도 허용합니다. 이를 간단히 '빈 커밋'이라고 합니다. 빈 커밋 작성 방법은 지금까지 살펴본 메시지 작성 방법과 약간 다릅니다.

4.7.1 세 번째 커밋

실습을 위해 index.htm 파일 내용을 좀 더 수정하여 세 번째 커밋을 하겠습니다. 간략하게 <title>~</title> 사이의 내용을 JINYGIT으로 수정한 후 저장합니다.

```
infoh@hojin MINGW64 /e/gitstudy04 (main)


$ code index.htm VS Code 실행
```

index.htm

```
<!DOCTYPE html>

<html>

<head>

    <meta charset="utf-8" />

    <meta name="viewport" content="width=device-width, initial-scale=1">

 <title>JINYGIT</title>

</head>

<body>

    <h1>hello GIT world!</h1>

</body>

</html>
```

파일을 수정한 후에는 반드시 다음과 같이 다시 수정된 파일을 스테이지 영역에 재등록합니다.

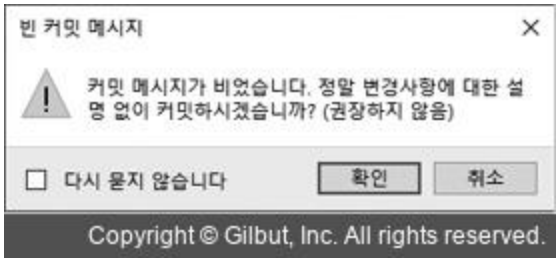
```
infoh@hojin MINGW64 /e/gitstudy04 (main)

$ git add index.htm  스테이지에 등록
```

4.7.2 소스트리에서 빈 커밋

소스트리에서는 메시지가 없는 빈 커밋을 하기가 더 쉽습니다. 예를 들어 파일 상태 탭을 선택한 후 아래쪽의 커밋 메시지 입력란을 비워 두고 커밋을 누르면 됩니다. 그러면 다음과 같은 확인 메시지가 표시됩니다. 확인을 누르세요. 이렇게 경고문을 알리는 것은 커밋 작업이 반드시 메시지를 작성하는 것을 원칙으로 하고 있기 때문입니다.

▼ 그림 4-24 빈 커밋 알림창



4.7.3 빈 커밋 확인

빈 커밋 메시지를 확인해 봅시다. 터미널에서 log 명령어를 입력합니다.

```
infoh@hojin MINGW64 /e/gitstudy04 (main)

$ git log 로그 확인

commit aa92947d350db27b604d1351930d4f809f96886e (HEAD → main)

Author: hojin <infohojin@gmail.com> 빈 커밋

Date:   Sat Jan 5 20:09:48 2019 +0900



commit aa1dd51a8883b2ea9a54209a00f434a2da01ee85

Author: hojin <infohojin@gmail.com>

Date:   Sat Jan 5 19:31:46 2019 +0900

    hello git world 추가



commit e2bce41380691b0a34aeab7db889a6c30fed8287

Author: hojin <infohojin@gmail.com>

Date:   Sat Jan 5 18:24:50 2019 +0900
```

인덱스 페이지 레이아웃

최신 커밋 로그를 보면 메시지 내용이 없는 것을 확인할 수 있습니다.

4.8 커밋 아이디

이번에는 커밋의 상세 내용을 확인해 보겠습니다. 다음과 같이 터미널에서 log 명령어를 실행하면 로그 정보를 볼 수 있습니다.

예

```
infoh@hojin MINGW64 /e/gitstudy04 (main)


$ git log


commit aa92947d350db27b604d1351930d4f809f96886e (HEAD → main) 아이디


Author: hojin <infohojin@gmail.com>


Date:   Sat Jan 5 20:09:48 2019 +0900


이하 생략
```

각 커밋에는 aa92947d350db27b604d1351930d4f809f96886e 같은 이상한 영문과 숫자가 있습니다. 이를 커밋 아이디라고 합니다. 커밋 아이디는 특정 커밋을 가리키는 절대적 이름이고, 명시적 참조 값입니다.

커밋 아이디는 다수의 커밋을 구분할 수 있는 키이며, 브랜치나 태그 등에도 많이 사용합니다. 이는 6장에서 브랜치를 설명할 때 더 알아보겠습니다.

4.8.1 SHA1

커밋 아이디가 이렇게 복잡한 영어와 숫자로 된 이유는 깃이 SHA1이라는 해시3 알고리즘을 사용하기 때문입니다. SHA1 해시키 값은 40자리의 복잡한 hexa 값으로 되어 있습니다. 깃은 스테이지 영역의 변경된 내용을 기반으로 SHA1 해시키를 생성합니다. 따라서 SHA1 해시는 중복되지 않은 고유의 키를 생성할 수 있는 장점이 있습니다.

깃이 SHA1 해시를 이용하는 것은 콘텐츠 추적과 분산형 저장 관리를 운영하면서 충돌을 방지하기 위해서입니다.

4.8.2 단축키

SHA1 해시키는 매우 복잡한 모양의 영어와 숫자로 되어 있습니다. 해시는 40자리의 16진수로 입력하다 실수로 잘못 입력할 가능성이 높습니다. SHA1 해시키는 매우 큰 숫자이기 때문에 고유 접두사로 간략하게 사용할 수 있는데, 해시의 앞쪽 7자만으로도 중복을 방지하면서 전체 키 값을 사용할 수 있습니다.

해시는 매우 큰 값으로 웬만해서는 앞쪽 숫자 값이 변경되는 경우가 드뭅니다. 물론 전체의 키 값을 다 사용하는 것이 좋겠지만, 앞자리 몇 개만 사용해도 충분합니다.

4.9 커밋 로그

깃은 터미널 기반의 응용 프로그램입니다. 앞서 커밋 목록을 나열하는 로그를 간단하게 살펴보았습니다. 깃의 로그는 저장소 커밋 기록들을 확인할 수 있습니다. 또 커밋 메시지, 아이디도 확인할 수 있고, 브랜치 경로 등을 분석할 수 있는 옵션들도 제공합니다.

4.9.1 간략 로그

커밋 메시지를 여러 줄 작성했다면 일반적인 로그 정보를 복잡하게 느낄 수 있습니다. 로그 옵션 중에서 --pretty=short를 사용하면 로그를 출력할 때 첫 번째 줄의 커밋 메시지만 출력합니다.

```
infoh@hojin MINGW64 /e/gitstudy04 (main)


$ git log --pretty=short 로그 확인


commit aa92947d350db27b604d1351930d4f809f96886e (HEAD → main)


Author: hojin <infohojin@gmail.com>




commit aa1dd51a8883b2ea9a54209a00f434a2da01ee85


Author: hojin <infohojin@gmail.com>


    hello git world 추가




commit e2bce41380691b0a34aeab7db889a6c30fed8287


Author: hojin <infohojin@gmail.com>


 인덱스 페이지 레이아웃
```

특정 커밋의 상세 정보도 확인할 수 있습니다. 특정 커밋의 상세 정보를 확인하고 싶다면 show 명령어를 사용합니다.

$ git show 커밋ID

4.9.2 특정 파일의 로그

전체 커밋과 달리 특정 파일의 로그 기록만 볼 수도 있습니다. 이때는 log 명령어 뒤에 파일 이름을 적어 주면 됩니다.

```
infoh@hojin MINGW64 /e/gitstudy04 (main)


$ git log index.htm 파일의 로그 확인


commit aa92947d350db27b604d1351930d4f809f96886e (HEAD → main)


Author: hojin <infohojin@gmail.com>


Date:   Sat Jan 5 20:09:48 2019 +0900
```

log 명령어의 옵션은 매우 다양합니다. 실습해 나가면서 다양한 옵션을 사용해 볼 것입니다. 소스트리를 이용하면 log 명령어와 복잡한 옵션을 사용하지 않아도 쉽게 로그 흐름을 파악할 수 있는 장점이 있습니다.

Note log 명령어의 여러 옵션

- -p 옵션: diff 기능(수정한 라인 비교)을 같이 포함하여 출력할 수 있습니다.
- --stat 옵션: 히스토리를 출력합니다.
- --pretty=oneline 옵션: 각 커밋을 한 줄로 표시합니다.

4.10 diff 명령어

diff 명령어는 커밋 간 차이를 확인합니다. 보통 리눅스나 macOS 같은 유닉스 계열의 운영 체제에는 유용한 diff 명령어가 있습니다. 깃 또한 초기 시작은 리눅스 커널을 개발하려는 것이었기 때문에 유사한 기능을 하는 diff 명령어를 제공합니다.

4.10.1 파일 간 차이

깃의 장점은 파일들의 수정 이력을 커밋이라는 형태로 구분할 수 있다는 것입니다. 앞에서 살펴보았듯이, 깃은 커밋으로 파일들의 수정 내역을 추적합니다. 파일 수정이란 파일 내용 일부가 수정, 추가, 삭제되는 것을 의미합니다. 개발하면서 수많은 소스 코드가 수정, 추가, 삭제되곤 합니다.

깃은 커밋을 기준으로 이러한 파일들의 수정 이력을 비교해 볼 수 있는 diff 기능을 제공합니다. diff 기능으로 파일의 수정 및 변경 내역을 쉽게 파악할 수 있습니다.

4.10.2 워킹 디렉터리 vs 스테이지 영역

아직 add 명령어로 파일을 추가하지 않은 경우, 워킹 디렉터리와 스테이지 영역 간 변경 사항을 비교할 수 있습니다.

index.htm 파일을 열어 <h1> 태그 밑에 <h2> 태그와 내용을 추가합니다.

```
infoh@hojin MINGW64 /e/gitstudy04 (main)


$ code index.htm
```

index.htm

```
<!DOCTYPE html>

<html>

<head>

    <meta charset="utf-8" />

    <meta name="viewport" content="width=device-width, initial-scale=1">

    <title>JINYGIT</title>

</head>

<body>

    <h1>hello GIT world!</h1>

    <h2>깃을 이용하면 소스의 버전 관리를 쉽게 할 수 있습니다.</h2>

</body>

</html>
```

4.10.3 커밋 간 차이

스테이지 영역에 있는 수정된 파일을 아직 커밋하지 않았다면, 최신 커밋과 변경 내용을 비교하여 볼 수 있습니다. HEAD는 마지막 커밋을 가지고 있는 포인터입니다.

워킹 디렉터리와 마지막 커밋을 비교해 봅시다.

```
infoh@hojin MINGW64 /e/gitstudy04 (main)

$ git diff head head 포인터 입력

diff --git a/index.htm b/index.htm

index f5097d9..56af0de 100644

--- a/index.htm

+++ b/index.htm

@@ -7,5 +7,6 @@

 </head>

 <body>

     <h1>hello GIT world!</h1>

+    <h2>깃을 이용하면 소스의 버전 관리를 쉽게 할 수 있습니다.</h2>

 </body>

 </html>

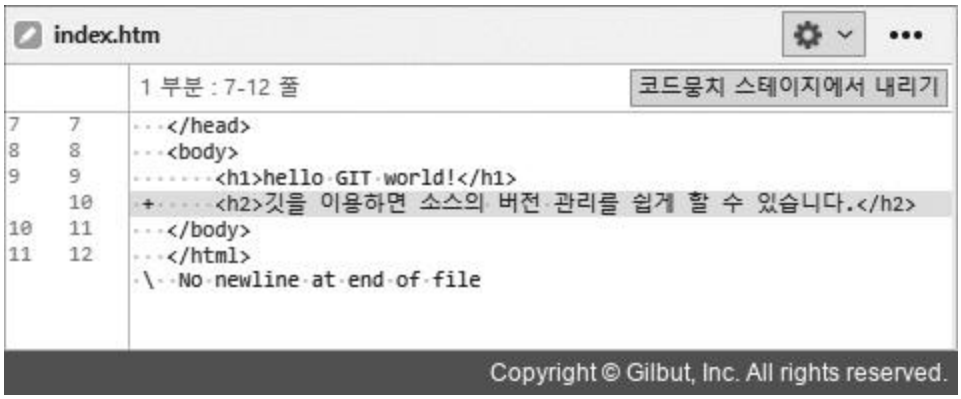
\ No newline at end of file
```

HEAD는 최근의 커밋 중 가장 마지막 커밋의 위치를 가리키는 값입니다. HEAD를 이용하면 최신 커밋과 이전 커밋을 비교하여 출력할 수 있습니다.

4.10.4 소스트리에서 간단하게 변경 이력 확인

소스트리를 이용하면 이를 좀 더 직관적으로 알아볼 수 있게 표현할 수 있습니다. 예를 들어 소스트리에서 수정한 파일 이름을 클릭하면 오른쪽 창에 파일 변경 내역들이 출력됩니다. 다음은 지금까지 실습한 index.htm의 변경 내역입니다.

▼ 그림 4-26 소스트리에서 변경 이력 확인 1



예를 들어 index.htm에서 <h1>hello GIT world!</h1>을 삭제한 후 다시 확인해 보면 삭제된 항목은 붉은색으로 표시합니다.

▼ 그림 4-27 소스트리에서 변경 이력 확인 2



- + 표시는 새롭게 추가된 내용을 의미합니다. 색상은 녹색으로 표시됩니다.
- - 표시는 삭제된 내용을 의미합니다. 색상은 붉은색으로 표시됩니다.

이러한 diff 기능은 나중에 자신이 수정한 부분들을 쉽게 찾을 수 있도록 도와줍니다. 또 다른 사람들과 협업하여 개발하는 과정에서 코드를 리뷰할 때 전체를 파악하는 것보다 수정된 부분만 확인하면 되므로 좀 더 쉽게 검토할 수 있습니다.

4.10.5 diff 내용을 추가하여 커밋

커밋 메시지를 작성할 때 -v 옵션을 같이 사용하면 vi 에디터에서 diff 내용을 추가할 수 있습니다.

예

```
infoh@hojin MINGW64 /e/gitstudy04 (main)


$ git commit -v diff 내용 추가
```

다음과 같이 커밋 메시지를 작성하면 diff 내용이 추가됩니다.

▼ 그림 4-28 커밋 메시지를 작성할 때 diff 내용 추가



Please enter the commit message for your changes. Lines starting

with '#' will be ignored, and an empty message aborts the commit.

#

On branch main

Changes to be committed:

modified: index.htm

#

```
# ------------------------------------------------------------

# Do not modify or remove the line above.

# Everything below it will be ignored.

diff --git a/index.htm b/index.htm

index 56af0de..a82537d 100644

--- a/index.htm

+++ b/index.htm

@@ -8,5 +8,6 @@

<body>

<h1>hello GIT world!</h1>

<h2>깃을 이용하면 소스의 버전 관리를 쉽게 할 수 있습니다.</h2>

<p>커밋 -v 옵션을 통하여 diff 내용을 추가할 수 있습니다.</p>

</body>

</html>

E:/gitstudy/.git/COMMIT_EDITMSG [unix] (02:40 14/01/2019) 1,0-1 꼭대기

"E:/gitstudy/.git/COMMIT_EDITMSG" [유닉스] 23L, 718C
```

4.11 정리

커밋 작업은 깃에서 소스 코드를 관리하는 첫 단추입니다. 너무 많은 코드를 수정한 후 커밋하는 것보다는 작은 단위로 코드를 수정한 후 커밋하는 것을 추천합니다. 커밋의 수정 부분이 적을수록 검토하기 쉽고, 오류도 쉽게 찾을 수 있습니다.

1 커밋 메시지를 좀 더 간단히 입력할 수 있는 방법은 바로 뒤에서 설명합니다.

2 4.4.3절에서 커밋을 해야만 “working tree clean” 메시지를 볼 수 있습니다.

3 해시 관련 설명은 이 책의 학습 사이트(https://git.jiny.dev)를 참고해 주세요.